Embedded Systems (M233)

Mock Midterm 2 — questions with complete solutions

100 points, two hours. QCM answers must be justified in one sentence each.

Part A — Quick check (8 × 2 pts = 16 pts)

Each statement is true or false. **Justify in one sentence.**

1. A pin configured `INPUT_PULLUP` reads `LOW` while the button connected to it is pressed.
2. A `delay(1000)` inside `loop()` prevents the sketch from seeing a button press that happens during that second.
3. `analogWrite(9, 128)` outputs a square wave of approximately 50% duty cycle on pin 9.
4. After `Serial.parseInt()` consumes the digits, a leftover newline stays in the buffer and is read by the next `Serial.read()`.
5. A switch wired with no pull resistor and left open leaves the pin floating, and `digitalRead` may return either value.
6. Calling `digitalWrite(pin, HIGH)` while the pin is still in its default `INPUT` mode has no effect at all.
7. `millis()` wraps around after about 49.7 days, and the test `millis() - previous >= interval` still works across the wrap.
8. Reading a pin configured as `OUTPUT` returns the last value written to it.

Part B — Wiring table (12 pts)

A prototype uses: a push button to GND on D2, a red LED with its series resistor on D8, a buzzer on D9 and a potentiometer on A0 (wiper), ends to 5V and GND.

(a) Complete the table: for each component give the pin, the correct `pinMode`, and the level that means *active*.

(b) The red LED drops 2 V and must run at 10 mA from a 5 V output. Compute the series resistor and give the nearest standard value.

(c) Why is the button wired to GND and read with `INPUT_PULLUP` rather than wired to 5V and read with plain `INPUT`? Give both reasons.

Part C — Code trace and bug hunt (14 pts)

Consider the sketch:

```
const int BTN = 2, LED = 8;
int count = 0;
void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
```

```
}
void loop() {
  if (digitalRead(BTN) == LOW) {
    count = count + 1;
    Serial.println(count);
    delay(400);
  }
  digitalWrite(LED, (count % 2 == 1) ? HIGH : LOW);
}
```

The intended behaviour is: each press prints the next integer and toggles the LED. The button is tapped at $t = 0.00\,\text{s}$, $0.20\,\text{s}$, $0.50\,\text{s}$, $0.70\,\text{s}$ and $1.00\,\text{s}$. Each of the first four taps is short (the contact is released well within 100 ms); the fifth press is **held down** from $t = 1.00\,\text{s}$ until $t = 2.00\,\text{s}$. Contact bounce is negligible.

(a) Give the exact list of values printed, with the approximate time of each print, and the LED level after each print.

(b) Name the two defects of this sketch with respect to the intended behaviour.

(c) Rewrite `loop()` so that the sketch counts each press exactly once, survives a held button, and never blocks.

Part D — Finite state machine: greenhouse vent (30 pts)

Design the controller of a greenhouse vent. The motor drives the vent; a full travel in either direction takes exactly 8 s. Inputs: `TEMP` (D3, active HIGH, hot) and `RAIN` (D4, active HIGH). Outputs: `DIR` (D7, HIGH = opening), `EN` (D8, motor enable) and a warning LED (D9) that must **blink at 1 Hz while the vent is moving** and stay OFF otherwise. Behaviour: from `CLOSED`, a hot reading starts opening. From `OPEN`, a cold reading or rain starts closing. A move always runs to completion (8 s). While opening, ignore further sensor changes; while closing, ignore everything as well. Rain arriving while the vent is opening lets the move finish, then the vent closes immediately.

(a) Give the state set, the full transition table (state, event, next state, outputs), and the state diagram.

(b) Write the complete sketch: non-blocking, one `enum class` state variable, no `delay()` anywhere, a 500 ms LED blink while moving, and the rain-while-opening rule of the spec.

(c) Trace the controller on the timeline: `TEMP` goes HIGH at $t = 0$, `RAIN` goes HIGH at $t = 3\,\text{s}$, `RAIN` goes LOW at $t = 20\,\text{s}$, `TEMP` goes LOW at $t = 21\,\text{s}$. Give the state, `DIR`, `EN` and the LED in the intervals $[0, 8)$, $[8, 16)$, $[16, 20)$, $[20, 28)$, and after 28 s.

Part E — Energy budget (14 pts)

A battery node runs from three AA cells in series (4.5 V, 2000 mAh). Every cycle of 10 s it wakes for 200 ms at 25 mA (radio send) and sleeps the remaining 9.8 s at 0.1 mA.

(a) Compute the average current per cycle in mA.

(b) Compute the battery lifetime in days.

(c) The cycle is stretched to 60 s with the same 200 ms wake. Recompute the average current and the lifetime. Which change dominates the gain?

Solutions

Part A

Solution – 1

True. With `INPUT_PULLUP` the pin is pulled to 5 V internally and the pressed button connects it to GND, so the pressed level is `LOW`.

Solution – 2

True. `delay` is blocking: nothing else runs, so a press during the second is not sampled and is lost.

Solution – 3

True. `analogWrite` is 8-bit PWM (0–255); 128/256 = 50% duty at about 490 Hz on the Uno.

Solution – 4

True. `parseInt()` consumes only the digits (and sign), leaving the newline in the buffer — the classic trap that makes the following `read()` return 10 immediately.

Solution – 5

True. Without a pull resistor the input is floating between two logic levels; its read value is undefined and can vary with ambient noise.

Solution – 6

False. On the AVR, writing `HIGH` to a pin in `INPUT` mode connects the internal pull-up resistor; the write does have an effect.

Solution – 7

True. `millis()` is a 32-bit unsigned count of milliseconds (2^{32} ms $\approx$ 49.7 days); unsigned subtraction makes the interval test correct across the wrap.

Solution – 8

True. Reading an `OUTPUT` pin returns the value currently driven on it (the port register), i.e. the last value written.

Part B

Solution – a

Component	Pin	`pinMode`	Active level
Button (to GND)	D2	`INPUT_PULLUP`	`LOW` (pressed)
Red LED + resistor	D8	`OUTPUT`	`HIGH` (lit)
Buzzer	D9	`OUTPUT`	`HIGH` (on)
Potentiometer wiper	A0	`INPUT` (analog)	value 0–1023

Solution – b

$R = (5-2)\,\text{V}/10\,\text{mA} = 300\,\Omega$; the nearest standard values are 300 Ω (E24) or 330 Ω, which gives about 9 mA — always round *up* to be safe.

Solution – c

(1) Safety: a wire to 5 V that comes loose reads as a press (false HIGH with a plain pull-down), while a loose GND wire just reads "not pressed". (2) Simplicity: the internal pull-up means no external resistor, and the pressed level being `LOW` is exactly what `INPUT_PULLUP` gives for free.

Part C

Solution – a

Tap at 0.00 is seen: prints **1**, LED ON, then the sketch blocks until 0.40. The 0.20 tap happens inside the delay and is *missed*. Tap at 0.50 is seen: prints **2**, LED OFF, blocked until 0.90. The 0.70 tap is missed. Tap at 1.00 is seen: prints **3**, LED ON, blocked until 1.40 — but the button is still held, so at 1.40 it prints **4** (LED OFF), at 1.80 prints **5** (LED ON), and it would keep auto-repeating about every 0.4 s for as long as it is held. Prints: 1, 2, 3, 4, 5, ...

Solution – b

Defect 1 (missed presses): the blocking `delay(400)` blinds the sketch, so fast presses are lost. Defect 2 (auto-repeat): the level test re-triggers as long as the button is held, so one long press counts many times. The intended "one count per press" needs edge detection plus a non-blocking debounce.

Solution – c

```
const int BTN = 2, LED = 8;
int count = 0;
bool lastPressed = false;
bool stable = false;
unsigned long tChange = 0;
const unsigned long DEBOUNCE = 25;

void loop() {
  bool raw = (digitalRead(BTN) == LOW);
  if (raw != lastPressed) { tChange = millis(); lastPressed = raw; }
  if (millis() - tChange >= DEBOUNCE && raw != stable) {
    stable = raw;
    if (stable) {                       // press edge only
      count = count + 1;
      Serial.println(count);
    }
  }
  digitalWrite(LED, (count % 2 == 1) ? HIGH : LOW);
}
```

The sketch never blocks; the debounce timer restarts on every bounce; the count fires only on the press *edge* (the `if (stable)` test), so holding the button counts exactly once.

Part D

Solution – a

States: `CLOSED`, `OPENING`, `OPEN`, `CLOSING`. Inputs: `TEMP`, `RAIN`; the move end is the 8 s timer.

State	Event	Next	DIR	EN	LED	Note
CLOSED	TEMP HIGH	OPENING	HIGH	HIGH	blink	
OPENING	8 s elapsed	OPEN	—	LOW	off	
OPENING	(other events)	OPENING	HIGH	HIGH	blink	ignored
OPEN	TEMP LOW or RAIN HIGH	CLOSING	LOW	HIGH	blink	
CLOSING	8 s elapsed	CLOSED	—	LOW	off	rain rule below
CLOSING	(other events)	CLOSING	LOW	HIGH	blink	ignored

Rain-while-opening rule: `OPENING` finishes its 8 s, enters `OPEN`, and the very same `loop()` pass sees `RAIN` (or `TEMP` low) true and moves straight to `CLOSING`. Diagram: `CLOSED` $\xrightarrow{\text{TEMP}}$ `OPENING` $\xrightarrow{8s}$ `OPEN` $\xrightarrow{\text{TEMP LOW} \cup \text{RAIN}}$ `CLOSING` $\xrightarrow{8s}$ `CLOSED`, self-loops "ignore" on `OPENING`/`CLOSING`.

Solution – b

```
enum class Etat { CLOSED, OPENING, OPEN, CLOSING };
Etat etat = Etat::CLOSED;
const int TEMP = 3, RAIN = 4, DIR = 7, EN = 8, LED = 9;
const unsigned long TRAVEL = 8000, BLINK = 500;
unsigned long tMove = 0, tBlink = 0;
bool ledOn = false;

void setup() {
  pinMode(TEMP, INPUT); pinMode(RAIN, INPUT);
  pinMode(DIR, OUTPUT); pinMode(EN, OUTPUT); pinMode(LED, OUTPUT);
}
void goMove(Etat e, int dir) {
  etat = e; tMove = millis();
  digitalWrite(DIR, dir); digitalWrite(EN, HIGH);
}
void stopMove(Etat e) {
  etat = e;
  digitalWrite(EN, LOW); digitalWrite(LED, LOW); ledOn = false;
}
void loop() {
  unsigned long now = millis();
  bool temp = digitalRead(TEMP) == HIGH;
  bool rain = digitalRead(RAIN) == HIGH;

  switch (etat) {
    case Etat::CLOSED:
      if (temp) goMove(Etat::OPENING, HIGH);
      break;
    case Etat::OPENING:
      if (now - tMove >= TRAVEL) {
        stopMove(Etat::OPEN);
        if (rain || !temp) goMove(Etat::CLOSING, LOW);   // rain rule
```

```
      }
      break;
    case Etat::OPEN:
      if (!temp || rain) goMove(Etat::CLOSING, LOW);
      break;
    case Etat::CLOSING:
      if (now - tMove >= TRAVEL) stopMove(Etat::CLOSED);
      break;
  }
  bool moving = (etat == Etat::OPENING || etat == Etat::CLOSING);
  if (moving) {                          // 1 Hz blink, non-blocking
    if (now - tBlink >= BLINK) { tBlink = now; ledOn = !ledOn; }
  } else { ledOn = false; }
  digitalWrite(LED, ledOn ? HIGH : LOW);
}
```

No `delay()`: travel and blink are two independent `millis()` timers; the rain rule is applied the instant the opening move ends, before any other transition.

Solution – c

Interval	State	DIR	EN	LED
$[0, 8)$	OPENING	HIGH	HIGH	1 Hz blink
$[8, 16)$	OPENING $\rightarrow$ CLOSING at $t = 8$ (rain seen at $t = 8$)	LOW	HIGH	1 Hz blink
$[16, 20)$	CLOSING	LOW	HIGH	1 Hz blink
$[20, 28)$	CLOSED (since $t = 16$)	—	LOW	off
after 28	CLOSED	—	LOW	off

At $t = 8$ the opening ends and `RAIN` is already HIGH, so the vent closes immediately (the rain rule). The closing finishes at $t = 16$, the controller is `CLOSED`; the later changes of `RAIN` ($t = 20$) and `TEMP` ($t = 21$) only matter if the vent would open again: at $t = 21$, `TEMP` goes LOW so nothing happens. Note $[20, 28)$ in the table collapses with "after 28": the vent has been closed since $t = 16$.

Part E

Solution – a

Charge per cycle: $25\,\text{mA} \times 0.2\,\text{s} + 0.1\,\text{mA} \times 9.8\,\text{s} = 5 + 0.98 = 5.98\,\text{mA}\cdot\text{s}$ over $10\,\text{s}$, so

$$I_{avg} = \frac{5.98\,\text{mA}\cdot\text{s}}{10\,\text{s}} = 0.598\,\text{mA} \approx 0.6\,\text{mA}.$$

Solution – b

$$T = \frac{2000\,\text{mAh}}{0.598\,\text{mA}} \approx 3344\,\text{h} \approx 139\ \text{days}.$$

Solution – c

Charge per 60 s cycle: $5 + 0.1 \times 59.8 = 10.98\,\text{mA}\cdot\text{s}$, so $I_{avg} = 10.98/60 \approx 0.183\,\text{mA}$ and

$$T = \frac{2000}{0.183} \approx 10\,929\,\text{h} \approx 455 \text{ days}.$$

The gain comes almost entirely from the *sleep share*: the wake cost (5 mA·s) is fixed per cycle, and halving it would matter far less than stretching the sleep. In the limit of long cycles, $I_{avg} \to 0.1\,\text{mA}$ (the sleep floor), so the battery is capped at about $2000/0.1 = 20\,000\,\text{h} \approx 833$ days no matter how rare the sends become.